

JDBC Practice Delivery Checklist

This checklist is not a rubric or an evaluation contract with which you can accurately determine the grade of your delivery. It is simply a list of the most frequent errors in these deliveries. The intention is that you use it before uploading your project to virtual campus.

The error checks discussed below are some of the most frequently detected over several courses, but they are not all. Not having any of these does not guarantee that your delivery will be perfect, but it will be much better than if you do not check it.

Preconditions
Service layer interfaces have not been modified Original acceptance tests are compiled and executed correctly These preconditions must be met so that delivery is not discarded. Interfaces must be respected as they have been delivered. They constitute a contract between the customer and the service provider. Note that a script will copy the original interfaces and tests into your delivery and then run the tests. Therefore, the modifications you have made to your tests and interfaces will be lost and compilation and/or execution errors will be generated, which will invalidate the delivery. Before making your submission, check this, lest you have inadvertently introduced some change in the code of the tests or interfaces. Java version must be 24 Modify the pom.xml file so that the java version is 24.
Execution
App won't start The project does not compile, boot, or allow access to the main interface. Some mandatory use cases were not performed or do nothing Some of the use cases marked as required are not developed or do not perform any observable actions Execution is interrupted with exceptions The application throws errors or unhandled exceptions that prevent it from functioning normally in scenarios that it shouldn't. Works with bugs The program runs, but it has obvious errors in the logic or results that prevent the correct use of the main functionalities.
Code readability, maintainability, and clarity
Code style and readability recommendations are not followed The formatting, indentation, or organization of the code makes it difficult to understand or maintain. The rules indicated in the statements (queries.properties, gateway names, transaction script names, etc.) are not followed Incorrect file names such as queries.properties, gateways or transaction scripts, or project structure other than specified Overly complex or unintelligible methods Implementations with convoluted logic, lack of comments or lack of modularity that prevents their reading or review will be penalized. Delivered project contains warnings Projects that generate unjustified and uncorrected warnings when compiling will not be accepted.
Layer pattern implementation: Improper coupling between layers
Higher-layer dependencies exist A lower layer (for example, persistence or services) depends on classes or components defined in higher layers (such as the user interface or application logic), thus not respecting the architecture seen in class. Dependencies exist on other subsystems of the same layer One component directly accesses another module of the same level, breaking the

internal independence of the layer.
Layer Pattern Implementation: Liability Violation by Layer
Presentation code outside the UI layer Elements related to the interface or user interaction appear in other layers of the system. Business code outside the business layer Application logic or domain rules are implemented at either the presentation layer or the persistence layer. Persistence code outside the persistence layer Access to databases or queries outside of the components designated for this purpose.
Implementing the DTO Pattern
Business code or persistence in an assembler (S2.5, P21) Assemblers should be limited to object-to-object conversion and contain no business logic or data access operations. Unnecessary transformations between dto objects (S11) The redundant creation or conversion of DTOs that do not provide functional or structural value will be penalized. The dtos of the persistence layer are not a view of the database (P16) Record transfer objects must faithfully reflect the stored information, avoiding divergences between the persistent model and the actual data.
Implementation of the Façade pattern
Utility façade with logic, out of place (S1) The façade should act only as an entry point to the business layer, without containing additional logic.
Implementation of the Factory pattern
Command objects do not use the persistence factory to obtain gateways (S12) Each command must access persistence resources exclusively through the corresponding factory. Objects in the display layer do not use the service factory to obtain facades (A2) The client classes must obtain the facades through the defined factory, not instantiate them directly.
Implementing TS and Command Patterns
Transaction scripts does not implement Command interface (S3) Commands not executed using CommandExecutor (S4) All operations must be piped through the CommandExecutor to ensure consistency and traceability Gateway objects create and release connections and/or transactions Some commands do not conform to the Javadoc of the interface (S9)
Implementing the TDG Pattern
TDG classes with business logic TDGs must access to the corresponding table. TDG Interfaces Do Not Extend Gateway Interface TDG interfaces add methods like those that already exist in Gateway, or methods that are not implemented or used Each TDG class must implement a single, well-defined method for adding, deleting, or updating records. Any additional method that attempts to perform partial or redundant insert, delete, or update operations deviates from the intended pattern. TDG interfaces add overly simple information retrieval methods. Choosing the type of method in the TDG Option A – Specific method (e.g., checkWorkOrderIsFinished(id)): Suitable when only a simple condition needs to be checked. Avoid transformation between ResultSet and Record and improve query efficiency.

Option B – Generic method (e.g., findByld(workOrderId)): Recommended when more information is required from the object or when the associated business logic is complex or can change. It maintains business responsibility in the corresponding layer and favors the reuse of the object. In short: use a specific method if you only need a one-time piece of data and retrieve the entire entity if the condition is part of the business logic. TDG interfaces add overly general information retrieval methods. Don't implement just a generic findAll method on the gateway and then filter all the information at the service layer; This causes inefficiency and unnecessary overload on the application. Gateway returns Record with data outside your table Gateways manipulate multiple tables or manipulate a foreign table inconveniently A Gateway in table A can query other tables to decide if a row in table A is a candidate for return. But not for other things.
SQL Statements
Business logic in overly complex queries or queries There are queries or connection parameters not externalized to the property files
JDBC
Resources are not released or are done incorrectly
Exceptions
Methods in service classes throw SQL, Persistence, or Runtime exceptions Persistence Class Methods Throw BusinessException Persistence class methods throw SQLException or RuntimeException instead of PersistenceException catch BusinessException, SQLException o RuntimeException (except CommandExecutor) in service classes catch BusinessException, PersistenceException, RuntimeException, or Exception in some class in an action package (except code provided) catch RuntimeException, NullPointerException, Exception anywhere in the code except that provided. Clases de UI no pueden throw new BusinessException(), throw new SQL, throw new Persistence.
Exceptions
Basic java errors (non-private fields except in Dto, object comparisons with == instead of equals, etc.)